

2025-07-12-SaturDat

How to build the Obsidian vault for:

[[PHILOVOID_FINALIZED_CODE]]d

[[Vault_Build_Instructions]]

Vault_Build_Instructions.md

Mission Directive: Construct a Fully-Saturated, Fully-Portable Obsidian-Compatible Knowledge Graph

This document contains all necessary instruction layers for an LLM agent to generate a complete recursive knowledge graph vault, structured for Obsidian and portable across multiple external platforms.

LAYER 1: GOD-LEVEL VAULT DIRECTIVES

Foundational Structure, Ontological Logic, and Saturation Protocol

You must begin by strictly following the **GOD-LEVEL ITERATIVE DENSIFICATION DIRECTIVES**, which define the recursive architecture of the vault. These include:

- File + Folder hierarchy enforcement
- 3-tier hierarchy (5 Main, 25 Sub, 95 Details)
- Link rules (parent, sibling, cousin, transversal)
- Markdown structure (7 mandatory content blocks per file)
- YAML schema for all nodes
- Tagging conventions
- File naming logic (1.2.3_Descriptive_Title.md)
- Dataview compatibility
- Bi-directional semantic linkage
- Structural verification protocols
- Manual and automated edge count validation
- Git snapshot + timestamping
- Finalization rule: no new levels post-saturation

This layer defines the Vault as an ontologically valid graph. You must not proceed to export or embed unless this structure is complete, connected, and verified.

LAYER 2: PORTABLE INTEROPERABILITY EXPORT RULES

Cross-Platform Embedding, Visualization, and Transformation Directives

Once the vault structure is complete according to Layer 1, you must generate the following **exports**, stored in a folder `/02_Export/` :

1. `graph_structure.json`
2. All node IDs, titles, parents, children, tags, excerpted content, and links
3. `graph.graphml`
4. GraphML version for use in Gephi, Cytoscape, etc.
5. `mindmap.markmap.md`
6. Markdown for Markmap.js mindmap rendering
7. `index.html`
8. Vault browser: static HTML representation of all notes
9. Include embedded D3.js or Vis.js force-directed graph
10. `vault_structure.ipynb`
11. Jupyter Notebook with:
 - Vault overview
 - Graph visualizations (e.g., `networkx` , `pyvis`)
 - Table-based dataview equivalents using `pandas`
 - Embedded markdown previews of sample notes
12. `README.md`
13. Usage instructions for:
 - Obsidian
 - GitHub Pages
 - Markmap
 - Jupyter
 - Notion

All exports must be self-contained, use relative links only, and reflect identical node/link structures as the source vault.

FINAL EXECUTION INSTRUCTION

This is your composite invocation command. No deviation permitted.

Construct a **recursive, fully-saturated Obsidian-compatible knowledge graph** vault from a 3-level topic model (5 Main Topics, 25 Sub-Topics, 95 Specific Details), with the following conditions:

- Apply LAYER 1 to build the vault according to all God-Level rules
- Upon completion, apply LAYER 2 to generate all exports
- Output 125 `.md` files in proper folders with correct YAML and internal wikilinks
- Link every node with parent, sibling, cousin, and transversal edges
- Export: `.json` , `.html` , `.graphml` , `.md` , and `.ipynb` files
- Return final `.zip` archive of full vault:

```
/GCN_PortableGraphVault/
```

```
|—— 00_System/
```

```
|—— 01_Main/
```

```
|—— 02_Export/
```

```
|—— Archive/
```

```
|—— README.md
```

Do not hallucinate.

Do not summarize.

Do not skip any instruction.

Vault must be fully recursive and exportable.

STRUCTURE CHECKLIST

Before returning final archive, ensure:

- [] 125 Markdown files exist and are linked
- [] All files have full YAML and 7-section content blocks
- [] Edge count ≥ 375
- [] No orphan nodes in Graph View
- [] All children resolve to valid nodes

- [] JSON, GraphML, HTML, Markmap, and Jupyter files are complete
 - [] Git repository initialized and README written
-

CLOSING PHILOSOPHY

“A Vault is not a folder. It is a recursive cosmos.

Links are not references—they are causal expressions of ontology.

Every edge you create echoes into a graph of thought.”

Execute with reverence and rigor. Do not simplify. Do not compress. Build the vault as if it were a metaphysical event.
